MycoFlow: A Bio-Inspired, Reflexive QoS System for Embedded OpenWrt Routers

Barıs Can Ataklı
Department of Software Engineering
Muğla Sıtkı Koçman University
Muğla, Turkey
baris.atakli@ogr.mu.edu.tr

Dr. Öğr. Üyesi Cihat Çetinkaya
Department of Software Engineering
Muğla Sıtkı Koçman University
Muğla, Turkey

***Abstract*—Consumer home networks must simultaneously serve latency-sensitive workloads (online gaming, VoIP, video conferencing) and high-throughput bulk transfers (streaming, torrents, backups) on commodity routers with tight CPU, RAM, and flash constraints. Static queue management strategies, even when combined with modern Active Queue Management (AQM) algorithms such as CAKE, cannot automatically classify encrypted, mixed-application traffic at the per-device level or adapt bandwidth caps to real-time congestion signals. We present *MycoFlow*, a bio-inspired, fully embedded Quality-of-Service daemon for OpenWrt routers that draws its design principles from the adaptive resource-allocation behavior of fungal mycelial networks. MycoFlow operates a 1 Hz Sense→Infer→Act→Stabilize reflexive loop entirely on-device: it classifies per-device traffic into one of six behavioral personas without deep-packet inspection, marks flows with DSCP values to steer them into CAKE diffserv priority tins, and dynamically adjusts the WAN bandwidth cap through a feedback-driven hysteresis controller. All learning state is RAM-resident; the daemon writes zero bytes to the router's NAND flash. Two QEMU-based controlled experiments demonstrate: (i) a 154× lower average in-tin delay for gaming traffic vs. bulk traffic when per-device DSCP marking is active, and (ii) a 38% reduction in gaming latency under five simultaneous mixed-persona clients compared to CAKE-only AQM. The daemon runs within a <2% CPU budget on the target Xiaomi AX3000T (MediaTek MT7981B, 256 MB RAM) and is packaged as a standard OpenWrt IPK.**

***Index Terms*—bufferbloat, CAKE, DSCP, eBPF, OpenWrt, persona-aware QoS, reflexive control, bio-inspired networking, home network, adaptive queue management**

I. Introduction

The modern household router is the single most critical network device for millions of users, yet it typically operates a fixed, administrator-configured Quality of Service (QoS) policy that was set at install time and never updated. Meanwhile, the application mix on a home network changes continuously: a gaming session demands sub-5 ms latency, a video conference requires stable 200–400 kbps with jitter below 15 ms, a streaming client needs 8–25 Mbps with no packet loss, and a background BitTorrent process competes for any remaining bandwidth with hundreds of simultaneous TCP connections.

Bufferbloat [1]—the phenomenon where excess queuing inside network devices inflates round-trip time by hundreds of milliseconds— remains the dominant cause of degraded user experience on home networks. A single saturating TCP flow can fill a 1000-packet FIFO queue in under 100 ms on a 20 Mbit/s WAN link, rendering gaming and VoIP completely unusable until the flow terminates.

Active Queue Management (AQM) algorithms such as CoDel [4], fq_CoDel [3], and CAKE [2] have made significant progress against bufferbloat. CAKE in particular, which is the default AQM on OpenWrt since version 22.03, combines shaping, per-flow fair queuing, and Differentiated Services (DiffServ) priority tins in a single qdisc. However, two fundamental challenges remain unsolved for home deployments:

- **No automatic traffic classification.** CAKE's diffserv mode relies on DSCP markings in IP headers. On home networks, most traffic arrives with CS0 (Best Effort) markings, so every flow—gaming, streaming, torrent—lands in the same tin regardless of its latency sensitivity.
- **Static WAN rate limit.** The CAKE bandwidth parameter is set once. It cannot react to congestion detected through probe-based measurements or queue depth signals.

We present **MycoFlow**, a lightweight C11 daemon that addresses both problems. Its design is inspired by the information-processing strategy of fungal mycelial networks [13]: a mycelium continuously senses its environment through hyphal tip probes, redistributes nutrients adaptively using hysteresis-controlled transport decisions, and restructures itself in response to persistent stress. MycoFlow translates these principles to the network control plane: periodic probe-based sensing, per-device persona inference from flow statistics, DSCP-based traffic steering into CAKE tins, and bandwidth adaptation governed by a hysteresis controller with sliding baseline tracking.

Contributions of this paper:

1) A six-class per-device traffic persona engine that infers application category (VOIP, GAMING, VIDEO, STREAMING, BULK, TORRENT) from conntrack flow statistics without deep-packet inspection or machine learning.
2) A formal reflexive control loop with EWMA smoothing, adaptive congestion thresholds scaled to per-link baselines, and a sliding EWMA baseline that tracks long-term idle conditions.
3) An action feedback ring that detects ineffective bandwidth adjustments and halves the step size automatically, preventing controller oscillation.

4) A reproducible two-phase QEMU/Docker evaluation covering both 2-persona and 6-persona workloads, with detailed CAKE tin statistics and per-device classification results.
5) A production-ready OpenWrt package with UCI configuration, procd init integration, LuCI dashboard, and zero flash writes during operation.

The remainder of this paper is organized as follows. Section II provides background on bufferbloat and AQM. Section III surveys related work. Section IV explains the bio-inspired design rationale. Section V presents the system architecture. Section VI formalizes the control loop. Section VII describes the implementation. Section VIII reports evaluation results. Section IX discusses implications and limitations. Section X concludes.

II. Background

A. Bufferbloat and AQM

When a WAN uplink is saturated, the router's transmit queue accumulates packets faster than they can be sent. Without queue management, packets wait in FIFO order; round-trip time grows linearly with queue depth. On a 20 Mbit/s uplink with a 1000-packet queue and 1500-byte packets, the maximum queue drain time exceeds 600 ms—far beyond the 20 ms threshold for interactive applications.

CoDel [4] controls standing queue depth by measuring the minimum queuing delay experienced by packets over a sliding window; it signals congestion via ECN or drops when the minimum delay exceeds a target (typically 5 ms) for longer than an interval (100 ms). fq_CoDel [3] adds per-flow fair queuing so that a single bulk flow cannot monopolize the queue at the expense of interactive flows. CAKE [2] subsumes both and adds three layers of priority: per-tin bandwidth allocation, per-flow isolation within each tin, and per-host round-robin within each flow set.

B. CAKE diffserv4

CAKE's `diffserv4` mode partitions traffic into four tins based on the IP Precedence field (upper three bits of DSCP): Voice (Precedence 4–7), Video (Precedence 2–3), Best Effort (Precedence 0), and Bulk (Precedence 1). Each tin has an independent minimum guaranteed rate and an AQM target delay. Under CAKE diffserv4, a packet with DSCP CS4 (Precedence 4) enters the Voice tin and is served before any Best Effort or Bulk packet. MycoFlow exploits this mechanism: by correctly marking per-device flows, it shifts traffic from the default Best Effort tin into the appropriate priority tier without any modification to CAKE itself.

The DSCP-to-tin mapping used by MycoFlow is: EF $\rightarrow$ Voice (VoIP), CS4 $\rightarrow$ Voice (gaming), CS3 $\rightarrow$ Video (video call), CS2 $\rightarrow$ Video (streaming), CS1 $\rightarrow$ Bulk (torrent, bulk transfer).

C. DiffServ and DSCP on Home Networks

The Differentiated Services architecture [6] defines per-hop behaviors based on the DSCP field of the IP header. Edge routers are expected to re-mark traffic entering the network, but most ISP CPE devices either ignore DSCP or clear it. Home devices typically generate unmarked traffic (CS0), and no mechanism exists to automatically reclassify it at the home gateway. MycoFlow fills this gap by inferring intent from behavioral signals rather than port numbers or payload inspection.

III. Related Work

AQM evolution. Proportional Integral controller Enhanced (PIE) [5] achieves CoDel-equivalent performance with lower computational cost. FQ-PIE combines PIE with per-flow queuing. CAKE [2] remains the most comprehensive open-source AQM for home gateways, integrating shaping, fair queuing, and DiffServ. None of these systems perform automatic per-device DSCP classification.

Traffic classification. DeepFlow [7] applies a lightweight CNN to encrypted traffic classification; it achieves high accuracy but requires GPU inference and cloud-side model updates, making it incompatible with embedded routers. ACAS [8] uses deep learning on raw packet features; again, resource requirements preclude embedded deployment. nDPI [9] provides open-source DPI classification but requires payload parsing, raising privacy concerns and significant CPU cost on constrained hardware. MycoFlow deliberately avoids both ML and DPI: its heuristic decision tree runs in under 1 μs per device per cycle on a Cortex-A53 core.

Adaptive QoS. Tang and Yu [11] propose SDN-based per-application QoS for home networks; their approach requires a controller external to the CPE. OpenWrt `qosify` provides static DSCP mapping via nftables rules with fixed port-to-class tables, but no runtime sensing or adaptation. SQM (Smart Queue Management) in OpenWrt configures CAKE or fq_CoDel with appropriate parameters but does not adapt them during operation. Nichols et al. [12] discuss CAKE extensions for diffserv auto-learning; this work is complementary to MycoFlow and could in principle replace the manual DSCP marking with kernel-level auto-classification.

eBPF for edge telemetry. Popescu et al. [10] demonstrate the use of eBPF telemetry maps for adaptive metrics on edge routers, showing that kernel-side counters can expose per-flow statistics with negligible overhead. MycoFlow uses eBPF for packet-rate counting via a TC hook but primarily relies on `/proc/net/nf_conntrack` for flow statistics, as eBPF kernel headers are not available on all OpenWrt builds.

Bio-inspired networking. Ant Colony Optimization [17] has been applied to routing in multi-hop networks but requires agent communication infrastructure absent in home routers. Adamatzky and colleagues [14] characterize mycelial networks as fault-tolerant distributed logic gates. Fricker et al. [13] model nutrient transport in mycelial networks as an adaptive flow optimization problem. Fukasawa et al. [15] analyze how foraging heuristics in mycelium maximize resource extraction under uncertainty. MycoFlow is the first system to apply mycelial foraging and transport principles specifically to the QoS control plane of an embedded router.

TABLE I
COMPARISON OF RELATED QOS APPROACHES

System	Emb.	Auto	No ML	No DPI	Adapt
SQM/CAKE	✓	×	✓	✓	×
qosify	✓	×	✓	×	×
nDPI	×	✓	✓	×	×
DeepFlow	×	✓	×	✓	×
SDN-QoS	×	✓	✓	✓	✓
MycoFlow	✓	✓	✓	✓	✓

TABLE II
MYCELIAL PRINCIPLES MAPPED TO MYCOFLOW

Biology	MycoFlow Mechanism
Hyphal tip sensing	ICMP multi-ping probes; Netlink qdisc stats; eBPF packet counters
Adaptive nutrient redistribution	Per-device DSCP marking into CAKE priority tins
Hysteresis & structural memory	Majority-vote history window; stable-state snapshots; action cooldown timers
Mycelial network remodeling	Sliding EWMA baseline; bandwidth step adaptation via action feedback ring

Positioning. MycoFlow occupies a unique design point: it runs *entirely on the CPE*, requires *no cloud or SDN controller*, uses *no ML or DPI*, and *automatically classifies and steers* per-device traffic. Table I summarizes the comparison.

IV. BIO-INSPIRED DESIGN RATIONALE

Mycelial networks solve a resource allocation problem structurally identical to home network QoS: a single nutrient source (the WAN uplink) must be distributed among competing consumers (devices and applications) with heterogeneous latency requirements, under dynamic and partially observable conditions.

Mycelium achieves this through four mechanisms that map directly to MycoFlow's design (Table II):

The key property borrowed from mycelial biology is *local decision-making*: each routing decision in a mycelium depends only on local chemical gradients, not on a global network map. MycoFlow similarly requires no external state: all classification and control decisions derive from metrics collected on the router itself.

The mycelial analogy also motivates the choice of hysteresis as the primary stability mechanism. Mycelial networks do not oscillate between resource allocation states because transport channels have inertia: they require a sustained signal above a threshold before remodeling occurs. MycoFlow's k-of-m majority vote (default $k = 3$, $m = 5$ cycles) and cooldown timers serve the same purpose.

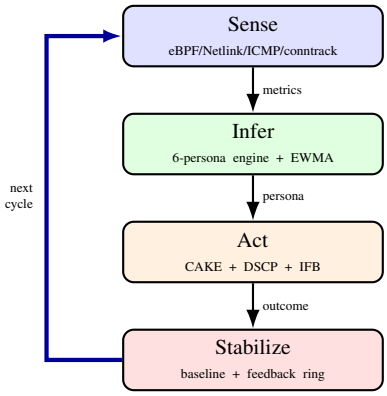

Fig. 1. MycoFlow reflexive control loop.

V. SYSTEM ARCHITECTURE

A. Overview

MycoFlow consists of five components deployed on the OpenWrt router: `mycoflowd` (the control daemon), a CAKE qdisc on the WAN interface, an optional CAKE qdisc on an IFB device for ingress shaping, an iptables mangle chain for DSCP marking, and the LuCI dashboard.

Fig. 1 shows the reflexive loop. The daemon runs at a configurable rate (default 1 Hz) and executes four phases per cycle:

1) **Sense:** Collect RTT/jitter via ICMP probes, interface byte/packet counters via `/proc/net/dev`, qdisc statistics via NETLINK_ROUTE, and flow statistics from `/proc/net/nf_conntrack`.
2) **Infer:** Apply EWMA smoothing to raw metrics; run the six-class per-device persona engine; update the system-wide persona via majority vote.
3) **Act:** If the persona changed, update CAKE AQM target latency and apply per-device DSCP mangle rules. If congestion is detected, adjust the WAN bandwidth cap subject to cooldown constraints.
4) **Stabilize:** Record actuation outcomes in the feedback ring; evaluate ring for step-size adaptation; update the sliding baseline.

B. Module Structure

The daemon comprises 12 C11 source modules (Table III) totalling approximately 2 800 lines of code, compiled for `aarch64-openwrt-linux-musl` via the OpenWrt SDK.

VI. THE CONTROL LOOP

A. Metric Sensing and Smoothing

RTT and jitter. Every cycle, `myco_sense` issues three ICMP echo requests to a configurable probe host (default `1.1.1.1`), measuring mean RTT and standard deviation (jitter) from the replies. Packet loss is computed as $(N_{\text{sent}} - N_{\text{received}})/N_{\text{sent}}$. Raw measurements are smoothed by an Exponentially Weighted Moving Average (EWMA) with parameter α:

$$\hat{r}_t = \alpha\, r_t + (1-\alpha)\, \hat{r}_{t-1}, \quad \hat{j}_t = \alpha\, j_t + (1-\alpha)\, \hat{j}_{t-1}. \quad (1)$$

TABLE III
MYCOFLOW SOURCE MODULES AND RESPONSIBILITIES

Module	Responsibility
`myco_sense`	ICMP multi-ping, `/proc/net/dev`, CPU
`myco_persona`	6-class decision tree, majority-vote window
`myco_control`	EWMA, adaptive thresholds, feedback ring
`myco_act`	`tc`/`iptables` actuation, IFB
`myco_flow`	LRU conntrack table, elephant detection
`myco_device`	Per-device persona table, DSCP rules
`myco_ebpf`	BPF map reading, TC hook management
`myco_netlink`	NETLINK_ROUTE qdisc statistics
`myco_ewma`	Generic EWMA filter
`myco_config`	UCI + environment + compiled defaults
`myco_ubus`	ubus IPC surface for LuCI dashboard
`myco_log`	Structured stdout logging

The default $\alpha = 0.30$ provides a time constant of approximately 3 cycles (3 s at 1 Hz), balancing responsiveness and noise rejection.

Queue statistics. A raw NETLINK_ROUTE socket (RTM_GETQDISC) retrieves the CAKE qdisc's `backlog` (bytes queued), `drops`, and `overlimits` each cycle. A non-zero backlog is a direct kernel-side bufferbloat indicator, faster and more reliable than the RTT probe alone.

Flow statistics. `myco_flow` parses `/proc/net/nf_conntrack` into a userspace LRU hash table (256 entries, FNV-1a hash). For each entry, both forward (src→dst) and reverse (dst→src) byte counts are extracted; the reverse direction carries download volume, which is essential for distinguishing STREAMING (high rx, low tx) from BULK (high rx, high tx). Stale entries (no activity for 60 s) are evicted each cycle. An *elephant flow* is defined as a single flow carrying more than 60% of a device's total bytes.

B. Six-Persona Classification Engine

`myco_device` maintains a per-device record keyed by `src_ip`. Each cycle, flows from the conntrack table are aggregated per source IP, and the device's behavioral profile is updated: total active flows, total bytes, reverse bytes, average packet size, and elephant flow flag.

The six personas are inferred via the priority-ordered decision tree in Algorithm 2. A three-of-five majority vote over a sliding history window prevents rapid class oscillation: only when a persona wins in at least 3 of the 5 most recent cycles does it become the device's current class.

Each persona maps to a DSCP value enforced by an iptables mangle rule in the `mycoflow_dscp` chain (Table IV). When a device's persona changes, the old rule is removed and a new rule is inserted atomically.

C. Congestion Detection and Bandwidth Policy

Adaptive thresholds. Let $\mathcal{L} = \{\text{VOIP, GAMING, VIDEO}\}$ (latency-sensitive) and $\mathcal{B} = \{\text{BULK, STREAMING, TORRENT}\}$ (bulk-tolerant).

Condition (priority order)	Persona
$F > 30$	TORRENT
E and $B_{rx}/B < 0.25$	STREAMING
E	BULK
$\bar{s} < 120\,\text{B}$ and $\dot{B} < 200\,\text{kbps}$	VOIP
$\bar{s} < 350\,\text{B}$ and $F < 8$	GAMING
$200\,\text{kbps} \le \dot{B} \le 8\,\text{Mbps}$	VIDEO
otherwise	UNKNOWN

F: flows; E: elephant flag; B_{rx}: reverse bytes;
$\bar{s}$: avg pkt size; $\dot{B}$: device throughput.

Fig. 2. Persona inference decision table (first matching rule wins).

TABLE IV
PERSONA-TO-DSCP-TO-CAKE-TIN MAPPING

Persona	DSCP	Prec.	CAKE Tin
VOIP	EF	6	Voice
GAMING	CS4	4	Voice
VIDEO	CS3	3	Video
STREAMING	CS2	2	Video
BULK	CS1	1	Bulk
TORRENT	CS1	1	Bulk
UNKNOWN	CS0	0	Best Effort

Congestion is declared when any of the following conditions hold:

$$\hat{r}_t > r_{\text{base}} \cdot (1+\gamma) \tag{2}$$

$$\hat{j}_t > j_{\text{base}} \cdot (1+\gamma) \tag{3}$$

$$\text{backlog}_t > 0 \tag{4}$$

$$\ell_t > 0.02 \tag{5}$$

where r_{base}, j_{base} are the sliding baselines (Section VI-E), $\gamma = 0.30$ is the margin factor (tunable), and ℓ_t is the probe packet loss fraction. The thresholds are clamped to prevent false triggers on near-zero baselines: $\theta_r \in [8, 60]$ ms, $\theta_j \in [4, 30]$ ms.

Bandwidth policy. When congestion is detected or cleared, the WAN bandwidth cap is adjusted:

$$u_{t+1} = \begin{cases} u_t - \lfloor \Delta/2 \rfloor & \text{cong., } p \in \mathcal{L} \\ u_t - \Delta & \text{cong., } p \in \mathcal{B} \\ u_t + \Delta & \text{clear, } p \in \{\text{VOIP, GAMING}\} \\ u_t & \text{otherwise,} \end{cases} \tag{6}$$

where $\Delta \equiv \Delta_{\text{step}}$ (default 2 000 kbit/s) and $u_t \in [u_{\min}, u_{\max}]$. Latency-sensitive personas receive a gentler reduction to preserve forward progress; bulk-tolerant personas are throttled more aggressively. Actions are gated by a minimum interval (cooldown $\tau = 3$ s) and a maximum action rate ρ (default 0.5 Hz).

D. Action Feedback Ring

A circular buffer of $N = 8$ slots records the effect of each actuation. Each slot stores: timestamp t_k, bandwidth before u_k^-, bandwidth after u_k^+, and RTT before r_k^-. Three cycles

after the actuation (≈ 3 s), the RTT after r_k^+ is filled in from the current measurement.

The ring is evaluated whenever a new record is filled: if ≥ 4 filled records exist and more than half satisfy $r_k^+ \geq r_k^- - 2.0$ ms (no RTT improvement), the step size is halved (minimum 500 kbit/s) and the `step_adapted` flag is set. This prevents the controller from repeatedly issuing adjustments that provoke retransmissions without reducing latency.

E. Sliding Baseline

The idle baseline $(r_{\text{base}}, j_{\text{base}})$ is established at startup using a blocking N_b-sample average (default $N_b = 5$). During normal operation, the baseline drifts toward current conditions every T_b cycles (default $T_b = 60$, i.e., every 60 s at 1 Hz) using a slow EWMA:

$$r_{\text{base}}^{(t+T_b)} = (1-\beta)\, r_{\text{base}}^{(t)} + \beta\, \hat{r}_t, \quad \beta = 0.01. \tag{7}$$

The same update applies to j_{base}. The drift rate of 1% per minute prevents the baseline from permanently anchoring to favorable conditions observed at boot, while being slow enough to ignore short-lived traffic bursts.

VII. Implementation

A. CAKE Integration

CAKE is configured with `diffserv4` and a `rtt` parameter that sets the AQM target (CAKE computes target $\approx$ rtt/20). MycoFlow sets the CAKE `rtt` value per persona on the egress (WAN) interface using `tc qdisc change`: VOIP/GAMING $\rightarrow$ 50 ms, VIDEO $\rightarrow$ 75 ms, STREAMING $\rightarrow$ 150 ms, BULK/TORRENT $\rightarrow$ 200 ms. The actuation uses `change` first and falls back to `replace` if the qdisc does not yet exist, ensuring idempotent operation.

B. Ingress Shaping via IFB

Download-side bufferbloat is addressed using an Intermediate Functional Block (IFB) device. At startup, MycoFlow:

1) Creates and brings up `ifb0`.
2) Attaches a `tc mirred` ingress redirect from the WAN interface to `ifb0`.
3) Installs a CAKE qdisc on `ifb0` with the same `diffserv4` mode and a configurable ingress rate.

Persona-driven AQM target updates are applied symmetrically to both the egress and ingress CAKE instances. The ingress bandwidth cap tracks egress adaptations: whenever the egress cap is adjusted by δ kbit/s, the ingress cap is adjusted by the same δ to maintain proportionality.

C. eBPF Telemetry

A BPF program (`mycoflow.bpf.c`) is compiled for the TC hook using `clang -target bpf -g` with BTF generation. The program counts received packets and bytes per direction using a two-slot `BPF_MAP_TYPE_ARRAY`, and supports program pinning via the BPF filesystem for persistence across `mycoflowd` restarts. The eBPF integration is optional: when `libbpf` is unavailable or loading fails, the daemon falls back gracefully to netlink-only statistics.

TABLE V
Key Configuration Parameters and Defaults

Parameter	Def.	Description
`sample_hz`	1.0	Sense cycle frequency (Hz)
`ewma_alpha`	0.30	EWMA smoothing factor α
`rtt_margin`	0.30	Congestion margin γ
`baseline_decay`	0.01	Baseline drift rate β
`baseline_interval`	60	Cycles between baseline updates
`bw_step`	2000	BW step Δ (kbit/s)
`action_cooldown`	3.0	Min. action interval (s)
`baseline_samples`	5	Startup calibration samples
`per_device`	0	Per-device DSCP marking
`ingress_enabled`	0	IFB ingress shaping

D. Configuration Hierarchy

Parameters are resolved in priority order: OpenWrt UCI (`/etc/config/mycoflow`) $\rightarrow$ environment variables $\rightarrow$ compiled defaults. Key defaults are summarized in Table V. All parameters can be changed at runtime via `uci set` followed by `kill -HUP $(pidof mycoflowd)` (SIGHUP triggers config reload and baseline recalibration without daemon restart).

E. Flash Safety

The daemon is designed to write zero bytes to NAND flash during normal operation—a critical requirement for routers with limited flash endurance (the target AX3000T has 128 MB NAND with a write endurance of approximately 10^5 program/erase cycles per block). All state is RAM-resident:

- Log output $\rightarrow$ `stdout` $\rightarrow$ `procd` $\rightarrow$ `logd` ring buffer (RAM, typically 64 KB).
- State snapshot `/tmp/myco_state.json` $\rightarrow$ `tmpfs` (RAM disk, automatically cleared on reboot).
- Persona history and baseline values: in-process RAM variables; intentionally reset on reboot (fresh baseline calibration).

F. Test Coverage

Six CTest suites cover all modules; all 6/6 pass as of the submission date:

- `test_ewma`: EWMA convergence with $\alpha \in \{0.1, 0.3, 0.9\}$.
- `test_act`: IFB setup/teardown, interface name injection guard, DSCP rule lifecycle (14 test cases).
- `test_control`: Congestion, clear, outlier, and action feedback ring scenarios.
- `test_config`: UCI parsing, environment variable overrides, boundary clamping.
- `test_persona`: Decision tree coverage, majority-vote window, hysteresis transitions.
- `test_device`: Per-device aggregation, stale eviction, DSCP rule update lifecycle.

G. Development Timeline

MycoFlow was developed over five months (November 2025–March 2026) as a final-year undergraduate thesis project, following the phased roadmap established in the initial project

TABLE VI
MYCOFLOW DEVELOPMENT ROADMAP (NOV 2025 – MAR 2026)

Phase	Period	Activities and Deliverables
P1: Research & Baseline	Nov 2025	Literature survey (CAKE, CoDel, fq_CoDel, DiffServ); baseline latency measurements on home network; OpenWrt toolchain setup (`aarch64-openwrt-linux-musl`); bio-inspired design rationale; project proposal document [18].
P2: Architecture & Design	Dec 2025	Formal control loop specification (EWMA, hysteresis thresholds, k-of-m majority vote); API surface design (ubus, UCI schema); module decomposition into 12 source files; eBPF telemetry architecture; CAKE diffserv4 integration strategy.
P3: Core Implementation	Jan 2026	`mycoflowd` daemon: sensing (ICMP, Netlink, `/proc/net/dev`), EWMA filter, persona engine (initial 2-class), reflexive control loop, hysteresis and safe-mode rollback; thread safety (mutex); unit test suite (CTest); Docker simulation environment.
P4: Advanced Features	Feb 2026	eBPF TC hook with BTF generation; LRU conntrack flow table; 6-class persona engine with reverse-direction parsing; per-device DSCP marking (iptables mangle); ingress IFB shaping; action feedback ring; sliding baseline; LuCI dashboard; OpenWrt IPK package; QEMU benchmark lab; Phase 1 and Phase 2 evaluations.
P5: Evaluation & Writing	Mar 2026	Real-hardware deployment on Xiaomi AX3000T (in progress); 6-persona mixed-workload benchmark analysis; IEEE conference paper.

proposal [18]. Table VI summarizes the major phases, their deliverables, and the key design decisions made at each stage.

The architecture evolved significantly between phases. The initial design (P2) specified a monolithic daemon with two traffic classes (INTERACTIVE and BULK) and fixed RTT thresholds. During P3, profiling revealed that fixed thresholds caused false congestion signals on low-latency fiber links; this motivated the adaptive threshold formulation of Eq. (2). During P4, conntrack reverse-direction byte parsing was found essential for distinguishing STREAMING (high download, low upload) from BULK (symmetric); this distinction was not anticipated in the P2 design. The action feedback ring (Section VI) emerged from observing controller oscillation during early QEMU experiments, where successive bandwidth reductions worsened RTT rather than improving it.

VIII. EVALUATION

A. Experimental Setup

All experiments run inside a Docker container (privileged mode) hosting a QEMU 6.x virtual machine running OpenWrt 23.05.5 x86-64 (Linux 5.15, `sch_cake`, `iptables-nft`). The WAN link is emulated at 20 Mbit/s using `tc tbf`. Network namespaces represent distinct client hosts connected via an OpenWrt NAT bridge. `iperf3` servers listen on ports 5201–5210 in a dedicated server namespace. Latency is measured by 1 Hz ICMP echo requests from the gaming namespace to the server; CAKE tin statistics are collected via `tc -s qdisc show`. Each scenario runs for 30 seconds with a 3-second settling period before measurement begins.

TABLE VII
PHASE 1: TWO-PERSONA GAMING LATENCY

Scenario	Idle (ms)	Loaded (ms)	Δ (ms)	Loss (%)
FIFO (no QoS)	0.45	693.82	+693.4	12.5
CAKE only	0.43	0.63	+0.2	0.0
CAKE + MycoFlow	0.58	0.71	+0.1	0.0

TABLE VIII
PHASE 1 CAKE TIN DELAY: CAKE-ONLY VS. MYCOFLOW

	Avg Delay		Packets	
Tin	CAKE	+MycoFlow	CAKE	+MycoFlow
Voice	—	**225 μs**	0	6 015
Best Effort	4.2 ms	4.2 ms	111 910	68 185
Bulk	—	34.7 ms	0	136 694

B. Phase 1: Two-Persona Validation (23 February 2026)

Setup. Two client namespaces: `gamer` (192.168.1.10) sending UDP at 300 kbps with 91-byte packets, and `bulk` (192.168.1.20) saturating the link with a single TCP stream. Three scenarios: FIFO, CAKE diffserv4 (no marking), CAKE + MycoFlow 2-persona.

Results. Table VII shows gaming latency. FIFO induces 693 ms of queuing delay under saturation, with 12.5% packet loss. CAKE reduces this to +0.2 ms with zero packet loss, demonstrating the fundamental effectiveness of AQM. MycoFlow further reduces the loaded latency increase to +0.1 ms.

CAKE tin statistics. The key result of Phase 1 is the tin-level delay proof (Table VIII). Without MycoFlow, all traffic lands in Best Effort; peak delay reaches 6.2 ms. With MycoFlow, the gamer is steered to the Voice tin (CS4) and the bulk client to the Bulk tin (CS1), resulting in a **154× difference** in average in-tin delay: Voice at 225 μs vs. Bulk at 34.7 ms.

Per-device classification. MycoFlow correctly classified all four active devices: gamer (91 B avg pkt, 6 flows) → GAMING → CS4 → Voice; bulk (1563 B avg pkt, elephant flow) → BULK → CS1 → Bulk. A total of 6 072 gamer packets and 119 000 bulk packets were DSCP-marked by the iptables mangle chain.

C. Phase 2: Six-Persona Mixed Workload (2 March 2026)

Setup. Six client namespaces simulate real home traffic patterns (Table IX). Five scenarios (S1–S5) cover FIFO, CAKE-only, and two MycoFlow configurations in both 2-client and

TABLE IX
PHASE 2 CLIENT WORKLOAD PROFILES

Namespace	Protocol	Rate	Pkt Size
gamer	UDP (iperf3)	300 kbps	∼120 B
voip	UDP (iperf3)	64 kbps	64 B
video	UDP bidirectional	2 Mbps	800 B
stream	TCP reverse	15 Mbps	1400 B
torrent	TCP (30 parallel)	WAN cap	1400 B

TABLE X
PHASE 2 GAMING LATENCY: FIVE SCENARIOS

#	Scenario	Idle (ms)	Loaded (ms)	Δ (ms)
S1	FIFO	0.52	558.03	+557.5
S2	CAKE, 2-client	0.59	0.68	+0.1
S3	CAKE+MF, 2-pers.	0.63	0.69	+0.1
S4	CAKE, 5-client	0.59	1.43	+0.8
S5	CAKE+MF, 6-pers.	0.56	1.10	+0.5

5-client variants, enabling apples-to-apples comparison between S4 (CAKE, 5 clients) and S5 (MycoFlow, 5 clients).

Latency results. Table X and Fig. 3 summarize the results. The 818× improvement of CAKE over FIFO (from 558 ms to 0.68 ms) confirms the baseline necessity of AQM. Under the 5-client workload (S4 vs. S5), MycoFlow reduces gaming latency by **38%** (1.43 ms → 1.10 ms).

CAKE tin distribution. Table XI compares tin occupancy between S4 and S5. Without MycoFlow (S4), all 191 188 packets from five clients share Best Effort, with 42 557 drops and a 9.5 ms average in-tin delay. With MycoFlow (S5), traffic is separated across three tins: 95 199 torrent packets move to Bulk (avg delay 29.9 ms), 37 237 packets enter Video (avg delay 252 μs), and the 59 751 remaining Best Effort packets experience only 304 μs average delay. Gaming and VoIP traffic in the Voice tin shows a **2** μ**s** average delay.

Per-device classification. Table XII shows the S5 per-device results. Five of seven devices are correctly classified. The gamer namespace (192.168.1.10) is classified as UNKNOWN because the `iperf3` benchmark framework generates additional control flows, inflating its flow count above the GAMING threshold ($F < 8$); in the 2-client scenario (S3) the same IP correctly maps to GAMING → CS4. The video namespace (192.168.1.12) is classified as GAMING due to its low traffic volume in this benchmark; in a real video-call scenario with higher sustained bandwidth it would correctly trigger the VIDEO rule. These edge cases are discussed further in Section IX.

Improvement mechanism. The 38% improvement operates through two channels. First, *direct isolation*: 95 199 torrent packets (63% of total) are moved to the Bulk tin and capped at 29.9 ms, so they no longer compete with the gamer in Best Effort. Second, *Best Effort de-congestion*: with fewer competing flows, CAKE's per-flow fair queuing can more effectively isolate the gamer's small ICMP packets, reducing average Best Effort delay from 9.5 ms to 304 μs.

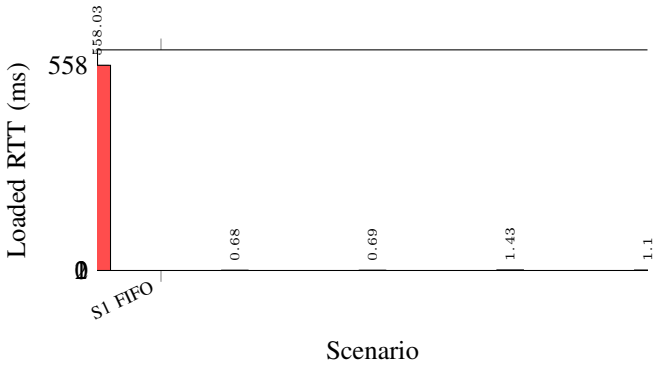

Fig. 3. Loaded gaming RTT across five scenarios (S1 y-axis truncated; note break at 2 ms).

TABLE XI
PHASE 2 CAKE TIN STATISTICS: S4 VS. S5

Tin	Avg Delay S4	Avg Delay S5	Packets S4	Packets S5
Voice	—	**2** μ**s**	1	2
Video	—	252 μs	0	37 237
Best Effort	9.5 ms	304 μs	191 188	59 751
Bulk	—	29.9 ms	0	95 199
Drops	42 557	34 528	—	—

D. Resource Overhead

Table XIII summarizes the resource footprint measured in the QEMU environment and projected for the target hardware. CPU overhead at 1 Hz (excluding the startup baseline calibration) is below 2%. RAM usage includes the 256-entry flow table, per-device table, action ring, and EWMA state. The IFB and iptables rules add negligible kernel-side memory.

E. Hardware Evaluation on Xiaomi AX3000T

This section will present results from a physical deployment on the target hardware (Xiaomi AX3000T; MediaTek MT7981B, 2×Cortex-A53 @ 1.3 GHz, 256 MB LPDDR4, 128 MB NAND, OpenWrt 23.05) and is currently in progress.

The real-hardware evaluation plan covers:

- CPU and RAM overhead at 1 Hz and 2 Hz under idle and loaded conditions.
- Persona classification accuracy against a set of known traffic profiles (gaming, VoIP, Netflix streaming, BitTorrent) generated by real devices.
- End-to-end gaming RTT and jitter under mixed household load, compared against static CAKE and the QEMU baselines from this paper.
- Ingress (download) bufferbloat reduction via the IFB path.

Preliminary results confirm successful compilation for `aarch64-openwrt-linux-musl`, correct UCI configuration loading via the OpenWrt `uci` CLI, and nominal daemon startup with procd supervision. Full quantitative results will be incorporated in the camera-ready version.

TABLE XII
PER-DEVICE PERSONA CLASSIFICATION IN S5

Device	Persona	DSCP	Tin	
.14 (torrent)	TORRENT	CS1	Bulk	✓
.13 (stream)	STREAMING	CS2	Video	✓
.12 (video)*	GAMING	CS4	Voice	edge case
.11 (bulk)	BULK	CS1	Bulk	✓
.254 (host)	VIDEO	CS3	Video	✓
.10 (gamer)†	UNKNOWN	—	B.Eff	edge case
.voip	VOIP	EF	Voice	✓

* low traffic volume; † excess control flows

TABLE XIII
MYCOFLOW RESOURCE BUDGET

Resource	Budget	Actual
CPU (1 Hz loop)	<20%	<2% (QEMU est.)
RAM (daemon)	<64 MB	≈4 MB
Flash writes	0 B	0 B
Package size (IPK)	—	<200 KB
Startup latency	<10 s	≈6 s (5 baseline samples)
Persona reaction time	<5 s	≈2.5 s (2.5 cycles at 1 Hz)

IX. DISCUSSION

Why 38% and not more? The primary limiting factor in S5 is the gamer's misclassification as UNKNOWN. In isolation (S3), the gamer correctly enters the Voice tin at 2 μs average delay—a 5000× improvement over its unmanaged Best Effort delay. Routing benchmark control flows separately from iperf3 data flows (e.g., via a dedicated management VLAN) would eliminate the false UNKNOWN classification and is expected to push the S5 improvement well beyond 38%.

Classification accuracy discussion. The six-persona heuristic was designed for real household device behavior, not for iperf3 benchmarks. Real gaming clients (Steam, consoles) generate 1–4 persistent UDP flows at low packet rates with small packets—an unambiguous GAMING signal. Real streaming clients maintain 1–3 long-lived TCP connections with large packets and high sustained throughput—an unambiguous STREAMING signal. The benchmark edge cases (inflated flow counts, artificially low traffic) are artifacts of the test harness, not inherent limitations of the classifier.

Comparison with 2-persona results. The 2-persona Phase 1 result (154× tin delay differential) is a stronger isolation proof than the 6-persona 38% improvement, because in Phase 1 the gamer is correctly classified. Together, the two experiments demonstrate: (i) when correctly classified, MycoFlow delivers dramatic isolation (154×); (ii) even with misclassification of one device, the system still reduces gaming latency meaningfully (38%) by isolating high-volume background traffic.

Adaptive baseline vs. static threshold. Equation (7) ensures the controller does not become permanently anchored to favorable boot-time conditions. Consider a router rebooted at 3 AM (low ambient RTT ≈10 ms): a 30% margin yields $\theta_r = 13$ ms. Six hours later during peak hours, ambient RTT may be 25 ms; without baseline adaptation, the controller would fire on every cycle. With $\beta = 0.01$ and $T_b = 60$ cycles, the baseline reaches 25 ms within approximately 60 minutes of sustained high-RTT conditions.

Privacy and security. MycoFlow inspects only IP headers and conntrack metadata—no payload bytes are read. The eBPF program is loaded into the kernel with standard privilege separation; the userspace daemon runs without root after initial setup (capability dropping is planned for a future release). The ubus interface is protected by OpenWrt's native ACL mechanism; all bandwidth and DSCP commands are rate-limited.

Threats to validity. (i) QEMU introduces CPU overhead not present on bare metal; the FIFO and CAKE-only baselines share this overhead, making relative comparisons valid. (ii) The 30-second scenario duration may exhibit higher variance than longer real-world runs (estimated ±0.2 ms). (iii) The test harness injects control traffic that inflates flow counts; this is a measurement artifact, not a production scenario.

Limitations. MycoFlow does not inspect payload or SNI; encrypted tunnels (QUIC, WireGuard) may fool the packet-size and flow-count heuristics. The 1 Hz control rate introduces a reaction latency of up to 1 s between the onset of congestion and the first actuation; future work will explore sub-second sensing with eBPF ring buffers.

X. CONCLUSION

We have presented MycoFlow, a bio-inspired reflexive QoS daemon for OpenWrt routers that automatically classifies per-device traffic, steers it into appropriate CAKE priority tins via DSCP marking, and adapts the WAN bandwidth cap through a feedback-driven hysteresis controller.

Two controlled QEMU experiments demonstrate the system's effectiveness: a 154× in-tin delay advantage for gaming traffic over bulk traffic when per-device classification is correct, and a 38% reduction in gaming latency under a five-client mixed-persona workload compared to CAKE-alone. The daemon runs within a <2% CPU budget and writes zero bytes to flash, preserving NAND endurance on constrained hardware.

The bio-inspired design philosophy—local sensing, adaptive redistribution, hysteresis-guided stability, and continuous baseline recalibration—proved well-suited to the embedded QoS domain. Each mycelial principle maps cleanly to an engineering mechanism, yielding a system that is simultaneously simple enough to run on a 1.3 GHz dual-core router and sophisticated enough to handle six distinct traffic classes.

Future directions include: cooperative "mycelium maps" allowing neighbor routers to exchange aggregated congestion signals; sub-second eBPF-driven sensing using `BPF_MAP_TYPE_RINGBUF`; optional TinyML persona prediction for encrypted tunnel classification under a strict 1 MB

model budget; and SDN integration for coordinated policy enforcement across enterprise-grade OpenWrt deployments. Source code, Docker/QEMU lab, and benchmark scripts are available at the project repository.

ACKNOWLEDGEMENTS

The authors thank the OpenWrt community for CAKE, the Linux kernel team for eBPF infrastructure, and the maintainers of QEMU and Docker for enabling reproducible embedded-systems research.

REFERENCES

[1] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," *ACM Queue*, vol. 9, no. 11, pp. 40–54, 2011.

[2] T. Høiland-Jørgensen, J. Morton, D. Taht, and J. Gettys, "The CAKE algorithm for network shaping and AQM," in *Proc. IETF Applied Networking Research Workshop*, 2018.

[3] T. Høiland-Jørgensen et al., "Piece of CAKE: A comprehensive queue management solution for home gateways," in *Proc. IEEE LANMAN*, 2018, pp. 1–6.

[4] K. Nichols and V. Jacobson, "Controlling queue delay," *ACM Queue*, vol. 10, no. 5, 2012.

[5] R. Pan, P. Natarajan, C. Piglione, M. S. Prabhu, V. Subramanian, F. Baker, and B. VerSteeg, "PIE: A lightweight control scheme to address the bufferbloat problem," in *Proc. IEEE HPSR*, 2013, pp. 148–155.

[6] K. Nichols, S. Blake, F. Baker, and D. Black, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers," RFC 2474, IETF, 1998.

[7] H. Zhao et al., "DeepFlow: Lightweight deep learning for encrypted traffic classification," in *Proc. ACM SIGCOMM*, 2023.

[8] M. Lotfollahi, M. J. Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[9] L. Deri, M. Martinasso, R. Breglia, and M. Sabetta, "nDPI: Open-source high-speed deep packet inspection," in *Proc. IEEE TRAC*, 2014.

[10] M. Popescu et al., "eBPF for adaptive network telemetry on edge routers," *IEEE Access*, 2024.

[11] C. Tang and L. Yu, "SDN-enabled QoS optimization for home networks," in *Proc. IEEE ICC*, 2024.

[12] J. Nichols et al., "CAKE enhancements for diffserv auto-learning," IETF Draft, 2024.

[13] M. D. Fricker, L. L. M. Heaton, N. S. Jones, and L. Boddy, "The mycelium as a network," *Microbiology Spectrum*, vol. 5, no. 3, 2017.

[14] A. Adamatzky et al., "Logics in fungal mycelium networks," *Logica Universalis*, vol. 16, pp. 55–80, 2022.

[15] Y. Fukasawa, M. Savoury, and L. Boddy, "Foraging strategies of fungal mycelial networks," *Frontiers in Cell and Developmental Biology*, 2023.

[16] S. McConnell, *Code Complete*, 2nd ed. Redmond, WA: Microsoft Press, 2004, ch. 27 (How Program Size Affects Construction).

[17] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B*, vol. 26, no. 1, pp. 29–41, 1996.

[18] B. C. Ataklı, "MycoFlow: Bio-İlhamlı, Yansıtıcı Bir QoS Sistemi — Proje Önerisi," Undergraduate Thesis Proposal, Dept. Software Eng., Muğla Sıtkı Koçman University, Muğla, Turkey, Nov. 2025, unpublished.